

Master's thesis

Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

September 12, 2026

Faculty of Science

University of Helsinki

Contact information

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki,Finland

Email address: info@cs.helsinki.fi
URL: http://www.cs.helsinki.fi/

HELSINGIN YLIOPISTO – HELSINGFORS UNIVERSITET – UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Study programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ohjaajat — Handledare — Supervisors

Prof. Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidoantal — Number of pages
Master's thesis	September 12, 2026	41 pages

Tiivistelmä — Referat — Abstract

Context. Radio access network (RAN) control planes are written mostly in C++ for performance, at a well-known cost in memory safety and concurrency. Rust promises comparable performance with compile-time safety, but its fit for this latency-critical setting is unproven.

Objective. The thesis compares modern C++ and Rust on a user equipment (UE) Context Management service modelled on a real Nokia 5G base-station control-plane component (gNB-CU-CP). Average-throughput comparisons leave four dimensions open: tail latency, the C++ defect classes Rust prevents at compile time, the overhead of its safety model, and the cost of incremental adoption.

Methods. One language-agnostic specification is frozen first, then implemented twice with matched algorithms, concurrency model, allocator, and build settings, and with forced-idiom gaps logged. A synthetic workload is increased to overload, both versions pass the same parity tests, and the evaluation adds fault injection and a foreign function interface (FFI) prototype.

Results. (1) Safe Rust rejects all seven injected memory-safety and data-race defect classes at compile time. In C++ the same seven build cleanly. They appear only at runtime, and only if a sanitiser catches them. (2) Seven further classes fail in both languages. On two of them C++ tooling reports what Rust does not. (3) C++ can call the Rust service through a thin C interface. All of the `unsafe` code stays there, and any extra time on a real signalling call is at most 10.91% of that call. (4) A full request spends only 0.250% on language work. Passing work between threads takes the rest, so that clock cannot attribute a gap to C++ or Rust. Timed on its own, the core is 1.6× faster in Rust. (5) Below saturation the paired test on the end-to-end median resolves 1.3 μs in Rust's favour. That gap is too small to bound a language effect. The Rust versions keep up with the offered load to 448 kilo-operations per second (kops/s), the C++ versions only to 392. Once neither can keep up, C++ still finishes 1.7× as much work as Rust in 20/20 interleaved repeats with both allocators.

Conclusions. The main cost of adopting Rust is engineering effort and the learning curve, not run-time performance at nominal load. The gain is freedom from data races, not from concurrency defects in general. A nominal-load test would not expose the gap.

ACM Computing Classification System (CCS)
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, incremental adoption

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — övriga uppgifter — Additional information

Software study track

Contents

List of Figures iii

List of Tables iv

List of acronyms v

1 Introduction 1

1.1 Motivation . . . 2
1.2 Industrial context . . . 3
1.3 Problem statement . . . 4
1.4 Research questions and hypotheses . . . 4

2 Background: Systems Languages for Latency-Critical Software 6

2.1 RAN control-plane systems and the gNB-CU-CP . . . 6
2.2 Predictability and the two languages . . . 7
2.3 Research gap . . . 7

3 Research Methodology 10

3.1 The UE Context Management subsystem . . . 10
3.2 Research approach . . . 11
3.3 Methods per research question . . . 12
3.4 Fairness as a controlled variable . . . 13
3.5 Statistical treatment . . . 13
3.6 Reproducibility and research artefacts . . . 14

4 Design and Implementation 15

5 Experimental Setup and Evaluation 17

5.1 Testbed setup . . . 17
5.2 Workload and measurements . . . 18
5.3 Results . . . 19
5.3.1 Below saturation (RQ1, RQ3) . . . 19

5.3.2 Past saturation (RQ1) 20
5.3.3 Resource use (RQ3) 21
5.3.4 Safety via fault injection (RQ2) 22
5.3.5 Migration (RQ4) 23

6 Discussion **25**
6.1 Summary of findings 25
6.2 Discussion on the research questions 25
6.3 Revisiting the hypotheses 27
6.4 Engineering guidelines for incremental adoption 28
6.5 Where this study sits 28
6.6 Ethical and professional reflection 29

7 Conclusion and Future Work **30**

Statement on the Use of AI Tools **32**

Bibliography **34**

List of Figures

2.1 Where the studied subsystem sits in the 5G NG-RAN 6
2.2 What the neighbouring studies control, and what they leave varying 8

3.1 UE-context lifecycle state machine 10

4.1 Shared-nothing, thread-per-shard architecture 15

5.1 The two workloads, and the lifecycle the mixed one walks 18
5.2 The load sweep: tail latency and achieved throughput 20
5.3 The end-to-end operation decomposed, against what the sweep can resolve 20
5.4 Divergence past saturation 21
5.5 Resource use, and the two effects inside peak RSS 22
5.6 Where the guarantee stops: the outcome matrix 23
5.7 The C++-to-Rust FFI migration seam 24

6.1 Where Rust's compile-time guarantee stops 26

List of Tables

3.1 How each research question is answered 12

5.1 Per-operation latency at the representative load 19

List of acronyms

Each acronym below is also written out where the text first uses it.

Acronym	Expansion
3GPP	3rd Generation Partnership Project, the body that specifies the 5G standards
ABI	Application binary interface
AI	Artificial intelligence
AI-RAN	An architecture in which AI and radio access network workloads share one accelerated infrastructure
API	Application programming interface
ASN.1	Abstract Syntax Notation One
CPU	Central processing unit
CVE	Common Vulnerabilities and Exposures
CWE	Common Weakness Enumeration
DPDK	Data Plane Development Kit
EU	European Union
FFI	Foreign function interface
FIFO	First in, first out
FNV-1a	Fowler–Noll–Vo hash function, variant 1a
GCC	GNU Compiler Collection
gNB	Next-generation NodeB, the 5G base station
gNB-CU	gNB central unit
gNB-CU-CP	gNB central unit control plane: the system class studied here
gNB-CU-UP	gNB central unit user plane
gNB-DU	gNB distributed unit
HDR	High dynamic range, of the latency histogram
IMT-2030	International Mobile Telecommunications-2030, the ITU framework for 6G
ITU	International Telecommunication Union
KPI	Key performance indicator
kops/s	Kilo-operations per second
KVM	Kernel-based Virtual Machine
MPSC	Multi-producer, single-consumer
NG-RAN	Next-generation radio access network
PHY	Physical layer
RAII	Resource Acquisition Is Initialization
RAN	Radio access network
RQ	Research question
RRC	Radio Resource Control
RSS	Resident set size
UE	User equipment, an end device such as a phone or a modem
UECM	UE Context Management, the subsystem reimplemented and measured here

1 Introduction

Mobile networks are moving from specialised hardware to software on ordinary servers. Functions that once ran on fixed-purpose, vertically integrated equipment are now re-implemented as programs on general-purpose machines. The radio access network (RAN), long the most hardware-centric part of the stack, is part of that shift.

In 5G, the base station, formally the next-generation NodeB (gNB), is split into a central unit and one or more distributed units. The central unit is split again into control-plane and user-plane functions [1, 2]. Open-RAN pushes the same idea further: the RAN becomes virtualised, multi-vendor software linked by open interfaces [3]. The control plane that results, the gNB central unit control plane (gNB-CU-CP), is a large, stateful, highly concurrent program. It tracks every connected device, each of which the standards call a user equipment (UE), runs Radio Resource Control (RRC) signalling [4], and coordinates mobility and handovers, all within strict latency limits.

Load at a busy site is high. A single site can run thousands of connection-management and mobility procedures per second, each of which has to complete within the millisecond-scale budget the 5G system sets for control-plane signalling [5]. A deployed control plane may track millions of active devices at once. As the RAN moves to cloud-native, disaggregated deployments, the correctness, reliability, and timing of this software increasingly set the quality of the whole network. An outage or a tail-latency spike here is not a local defect. Every device the site serves is affected [6, 7].

This control-plane software is, today, written mostly in C++. The choice is a practical one: C++ offers the low-level control, mature tooling, and performance that latency-critical systems demand. But that performance comes with a well-known cost. C++ places the burden of memory and concurrency management on the programmer, and decades of experience show that this burden is routinely mishandled, producing use-after-free errors, double-frees, data races, and related defects that are expensive to find, dangerous in production, and responsible for a large share of security vulnerabilities in industry practice [8, 9]. Rust has emerged as the first mainstream systems language to promise an escape from this trade-off: memory safety and data-race freedom enforced at compile time on machine-checked foundations [10, 11], with performance intended to be competitive with C++ [12]. Whether that promise holds for a telecom-grade, latency-critical control plane, where predictability under overload matters as much as raw speed, is not well understood. This thesis, carried out at Nokia, addresses the question directly: it de-

velops two functionally equivalent implementations of a representative RAN subsystem, a UE Context Management service modelled on a real Nokia gNB-CU-CP component, in modern C++ and in Rust, and compares them under a realistic synthetic signalling workload.

Three terms recur throughout. The *gNB-CU-CP* is the control-plane software this work is about. *UE Context Management (UECM)* is the specific subsystem reimplemented and measured. *Tail latency*, the p99, p99.9 and p99.99 percentiles rather than the mean, is the property that, more than average speed, decides whether such a system meets its deadlines [6, 7].

1.1 Motivation

Three trends make this comparison timely. The first is the cloud-native evolution of the RAN. Control-plane functions now run on general-purpose servers, so the software is longer-lived, more concurrent, and must meet its deadlines without the determinism of purpose-built hardware. Open-RAN disaggregation turns the base station into virtualised, multi-vendor software [3]. A latency regression or a memory-corruption defect is then a network-wide risk, and the language choice is an engineering decision.

The second trend is artificial intelligence (AI), which the network once used to tune itself and now carries as a workload in its own right [13]. International Mobile Telecommunications-2030 (IMT-2030), the 6G framework of the International Telecommunication Union (ITU), lists that second role as one of its usage scenarios [14]. The architecture forming around it is AI-RAN, in which AI and radio work share one accelerated server [15]. A control plane there no longer has the host to itself. It shares the central processing unit (CPU), memory, and cache with inference work it does not schedule. When that neighbour takes capacity, the control plane saturates sooner: it stops keeping up with demand at a lower load than it would alone. Chapter 5 finds the widest gap between the two languages past that point.

The third is the shift toward memory-safe languages. Large software organisations report that the majority of their serious security vulnerabilities stem from memory-safety errors, and that adopting memory-safe languages reduces them [8, 9, 16, 17]. Independent work argues that memory safety now needs standardised requirements if it is to be procured at all [18]. Cybersecurity agencies urge a move away from C and C++, and the European Union (EU) Cyber Resilience Act adds legal duties from September 2026 and December 2027 [19, 20, 21]. The Act names no programming language. Direct evidence for a RAN

control plane is still missing. Providing it is the motivation for this work.

1.2 Industrial context

This thesis is carried out at Nokia. The company is one of the small number of vendors that build, maintain, and evolve production RAN software end to end. The five largest suppliers took 96% of worldwide RAN revenue over the first three quarters of 2025, with concentration by Herfindahl–Hirschman Index at a ten-year high [22, 23]. The subsystem it studies, UECM, is modelled on a real gNB-CU-CP component rather than a textbook example.

In the architecture of the 3rd Generation Partnership Project (3GPP), the gNB-CU-CP is the control-plane half of the central unit. It hosts RRC and manages the UE context lifecycle [1, 4], under the millisecond-scale budgets the 5G system sets for control signalling [5]. Chapter 2 places it in the wider architecture. Because it is written in C++, it inherits that language's performance and its memory-safety and concurrency hazards. The three trends of Section 1.1 meet here. Software of this class runs on live operator networks. The language choice is a production concern.

UECM is small enough to reimplement faithfully and reason about in full. It still carries the properties that make control-plane code hard. The implementation follows a public, frozen specification, not proprietary code, so readers outside Nokia can reproduce the comparison.

A decision with a generational horizon. A language choice in software of this class outlives the generation it was made for. 3GPP has allocated Release 20 to the 6G studies and Release 21 to the normative work. The ITU asks for technology proposals in early 2029 and for specification submissions by mid-2030 [24]. That work covers migration from 5G and interworking with it. A control plane written now is maintained alongside the next generation, not retired. IMT-2030 still names security and resilience as a design principle and still carries the ultra-reliable low-latency scenario in strengthened form [14].

The measurements here use a current-generation subsystem and published requirements. No claim is made about requirements that have not been written.

A concrete failure mode. Consider a mobility storm in which many UEs hand over at once. A UE context freed on release while an in-flight signalling message still references it is a use-after-free. In C++ that compiles and may pass ordinary tests. It can then corrupt

another UE's state or crash the worker under high concurrency. The same defect can show up as a tail-latency spike instead of a crash. In safe Rust the design does not compile: ownership and borrowing reject the dangling reference. That is why this thesis measures both *safety* (which defects are prevented, and when) and *predictability* (tail latency under overload), not average throughput alone.

1.3 Problem statement

Two properties of this domain make the language choice consequential. First, manual memory and concurrency management in C++ exposes the software to defects such as use-after-free, double-free, data races, and iterator invalidation, which are silent at compile time and costly to find and fix at telecom scale. Second, what governs whether RAN deadlines are met is not average throughput but worst-case and tail behaviour under load and overload [6], a property that is easy to regress and hard to measure correctly. Rust promises to remove the first class of defect at compile time while claiming performance competitive on the second, but that claim rests mostly on general-purpose benchmarks rather than on stateful, concurrent, latency-critical telecom workloads.

Bringing these threads together, the concrete problem this thesis addresses is the following. Telecom vendors are under growing pressure, both technical and increasingly regulatory, to adopt memory-safe languages, yet for the RAN control plane they lack *direct, controlled* evidence on which to base that decision. It is not established whether a Rust implementation of a stateful, concurrent control-plane subsystem can match modern C++ in *tail latency and predictability under overload* (as opposed to average throughput), whether Rust's compile-time guarantees actually eliminate the specific defect classes that affect the existing C++ code, and what incremental adoption *costs* inside a large C++ codebase. The available evidence is largely general-purpose and average-case [12, 25], leaving these three questions (predictability, prevented-defect scope, and migration cost) open for a representative subsystem. Answering them under controlled, reproducible conditions is the problem this thesis sets out to solve.

1.4 Research questions and hypotheses

The thesis is organised around four research questions. They put latency predictability first, then defect scope, then resource cost, then incremental adoption:

RQ1 How do Rust and modern C++ compare in latency predictability (median, p99,

p99.9 and p99.99 tail latency, and the maximum) for a RAN control-plane workload under increasing load and overload?

RQ2 Which classes of memory-safety and concurrency defects that occur in C++ are eliminated at compile time in the Rust implementation, and at what stage and cost are the remaining ones detected?

RQ3 What is the overhead, if any, of Rust's safety model (bounds checks, ownership-driven copies, abstraction cost) on throughput, CPU utilisation, and memory behaviour?

RQ4 What are the practical implications (effort, foreign function interface (FFI) interoperability, and risk) of adopting Rust incrementally within an existing C++ RAN software ecosystem?

Three hypotheses are written down before any measurement.

H1 For this workload the two languages are performance-equivalent below saturation, with any difference only in the far tail.

H2 Safe Rust prevents at compile time the memory-safety and concurrency defect classes that C++ only tries to catch after the program is built.

H3 The extra time when C++ calls into Rust is small next to the cost of the operations themselves, so the real cost of adding Rust a piece at a time is engineering work, not run-time speed.

Chapter 6 revisits all three against the evidence.

The defect list for RQ2 includes classes safe Rust is expected to prevent and classes it is expected not to. The study can then find where the safety guarantee ends, instead of only showing that it holds.

A previous study already estimated that boundary from a catalogue, not from compiled programs. It took the Common Weakness Enumeration (CWE), a long list of bug types, and marked each type as one that safe Rust would prevent or would not. It then counted real vulnerabilities in 68 open-source C projects. 58.2% of those classified vulnerabilities sat in types that safe Rust would have made almost impossible [26]. That share is a best case: it assumes the new code stays in safe Rust. A type counts as prevented if the mapping says so, even if no compiler has rejected a program. RQ2 asks the same question of programs that are compiled and run. That is how it can say when the remaining types are caught, and what it costs to catch them.

2 Background: Systems Languages for Latency-Critical Software

2.1 RAN control-plane systems and the gNB-CU-CP

The next-generation radio access network (NG-RAN) is the setting for this comparison. Event rates are high: UEs move, and signalling can spike. Delay bounds are tight and must stay stable. The 5G service and RAN specs state those bounds as key performance indicators (KPIs) [5, 27]. The software lives for a long time and keeps growing. Many threads run at once. Here the worst case and the tail matter more than the mean. That is why this thesis measures the tail.

The gNB is split on two axes. The first split is central versus distributed. A central unit (gNB-CU) hosts the higher layers of the stack. One or more distributed units (gNB-DU) host the lower layers. Those layers sit close to the radio and are tight on time. The second split is control versus user plane, inside the central unit. It splits the control-plane function (gNB-CU-CP) from the user-plane function (gNB-CU-UP). These parts talk over standard links. NG goes toward the 5G core. F1 sits between the CU and the DU. E1 sits between the CU-CP and the CU-UP [1, 2, 28]. Open-RAN pushes the same links further. Each part can then come from a different vendor and run as software on ordinary machines [3]. Figure 2.1 places the part studied here, UE Context Management (UECM), inside the gNB-CU-CP. UECM owns the per-UE control-plane state. It also handles RRC signalling and mobility. That is why its correctness and timing stand in for the control plane as a whole.

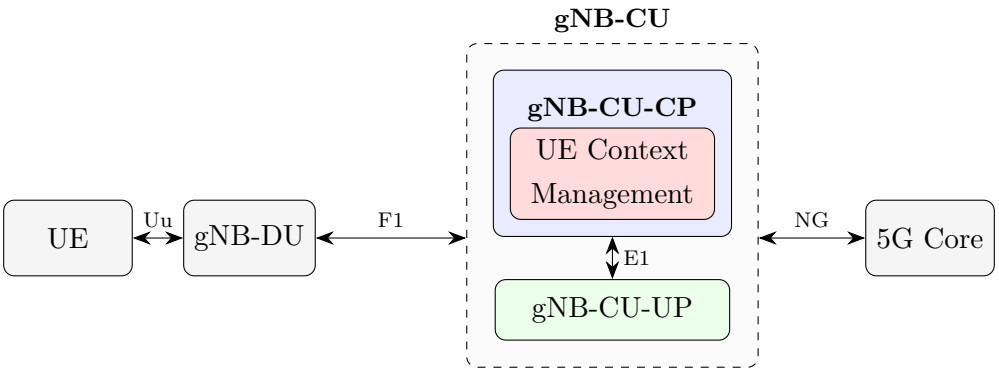


Figure 2.1: Where the studied subsystem sits in the 5G NG-RAN. UE Context Management is highlighted inside the gNB-CU-CP.

2.2 Predictability and the two languages

This section states what the study takes as given. Stable delay, not mean throughput, is the main property of a delay-critical service. So the quantity to measure is the tail, not the mean. Measuring the tail well also shapes the harness. The send schedule must be open-loop. A closed-loop sender stops while a stall is in progress. It then never records the delay of that stall.

C++ gives the control the domain needs. It leaves memory and thread safety to the programmer. Sanitisers and review stand in for a guarantee. The standards committee's work on enforced safety profiles is not a standard yet. It is not widely in use. Rust's ownership and borrowing rule out lifetime and aliasing bugs at compile time. Its `Send` and `Sync` traits rule out data races. That claim is narrower than "thread safety". It says nothing about lock order. It says nothing about whether a composed step is atomic. It says nothing about leaks. And it is off wherever a program writes `unsafe`. Both languages can express the same thread structures. A comparison that lets the structure change with the language then measures the structure, not the language. The two meet through a C application binary interface. That is the path any step-by-step migration has to use. It is also why a seam holds the `unsafe` a migration needs.

2.3 Research gap

The sections above set up the gap. The RAN control plane has many threads and tight delay bounds. Tail delay under too much load, not mean throughput, sets the quality the network delivers. C++ gives the control and speed the domain needs. But it leaves whole classes of memory and thread bugs to detection after the fact. Rust promises to block those classes at compile time at no run-time cost. It also offers a step-by-step adoption path through the C application binary interface (ABI).

The domain is not unstudied. The volume of prior work is easy to miss, because almost none of it is indexed. Five vendor-hosted master's theses, all done at Ericsson, have compared candidate languages for RAN and telecom software. One weighs run-time cost against how fast people can build, across C++, Rust, and Go [29]. One asks whether Rust could replace C in RAN node management [30]. One builds a packet processor, one thread per core, in async Rust. It measures that processor against the Data Plane Development Kit (DPDK) program it mirrors [31]. One measures median and p99 delay for control-plane steps across five async frameworks [32]. The last builds matched C++

and Rust versions of an Abstract Syntax Notation One (ASN.1) decompiler. It compares them with statistical tests [33]. All five are directly relevant. Two report gaps that do not favour Rust.

Three of them change the language together with the run-time and the thread model. A measured gap then cannot be blamed on the language rather than on the framework [29, 31, 32]. The effect sizes reach a factor of five. That shows how much of the measurement the run-time owns. The other two hold the design closer to fixed. But they report run time and peak memory, not tail delay under rising load [30, 33]. The closest of them, Emberg and Wadsten, measures a stateless transform. It is not a stateful multi-thread part. Figure 2.2 places the five vendor theses beside the general-purpose comparisons [34] and the matched systems-software pairs [35, 36]. The pattern is this. Each group holds some of what a fair test would need. None holds all of it.

That concentration is itself part of the gap. All five studies come from one vendor. Their conclusions have never been tested against another firm's software, targets or house rules. So what is known about this question is what one firm's frameworks say about it. The published record on the other side is thinner. There is no study of Rust among the indexed papers of the vendor hosting this work. The nearest peer-reviewed stance from that side argues for making memory-error sanitisation practical. It does not argue for replacing it with a compile-time guarantee [37]. That is the strategy this thesis tests against.

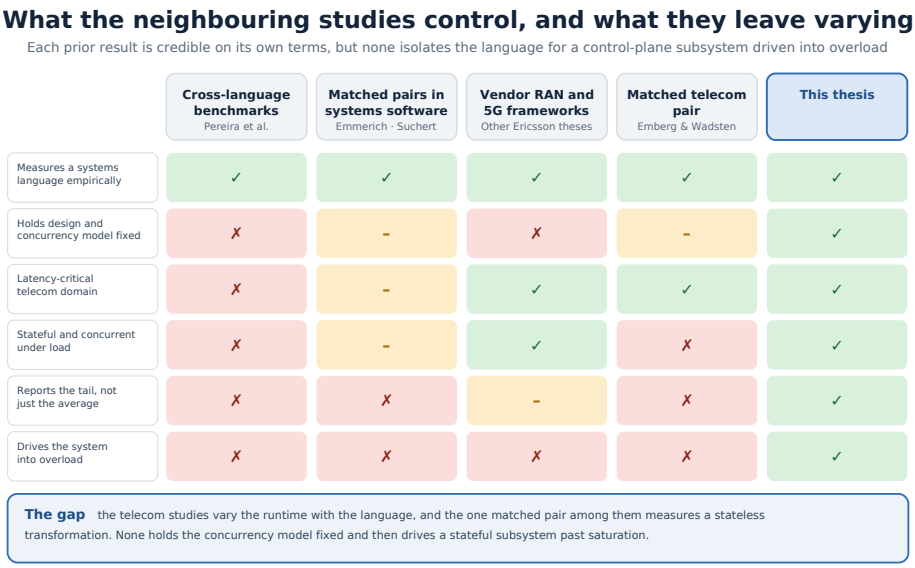


Figure 2.2: What the neighbouring studies control, and what they leave varying. Each row is a property a controlled answer would need, and each column is a group of studies. The vendor-RAN column is the Ericsson theses other than Emberg and Wadsten, who have their own column because they hold the design closer.

What is missing from prior work is a direct, controlled test of a typical control-plane part. The open questions are these. Does Rust hold its tail delay under too much load against modern C++ when the thread model, operations, and data layout stay fixed? Which bug classes does it remove, and at what stage are the rest caught? What does its safety model cost in throughput and in CPU and memory? What does step-by-step adoption cost in practice? Closing this gap is the purpose of the four research questions. The next chapter grounds them in the part under study, the bug classes, and the timing at stake. It also sets out the method.

3 Research Methodology

The last chapter set out the gap. This chapter says how the study closes it.

3.1 The UE Context Management subsystem

The part under study is a UE Context Management service. It is modelled on a real Nokia gNB-CU-CP service that owns the UE context lifecycle, RRC procedure execution [4], and mobility and handover. Those are the functions the 3GPP design assigns to the control-plane part of a split gNB (Section 2.1). Figure 3.1 shows the UE-context lifecycle both versions encode the same way. The mixed workload of Section 5.2 walks each UE through that machine.

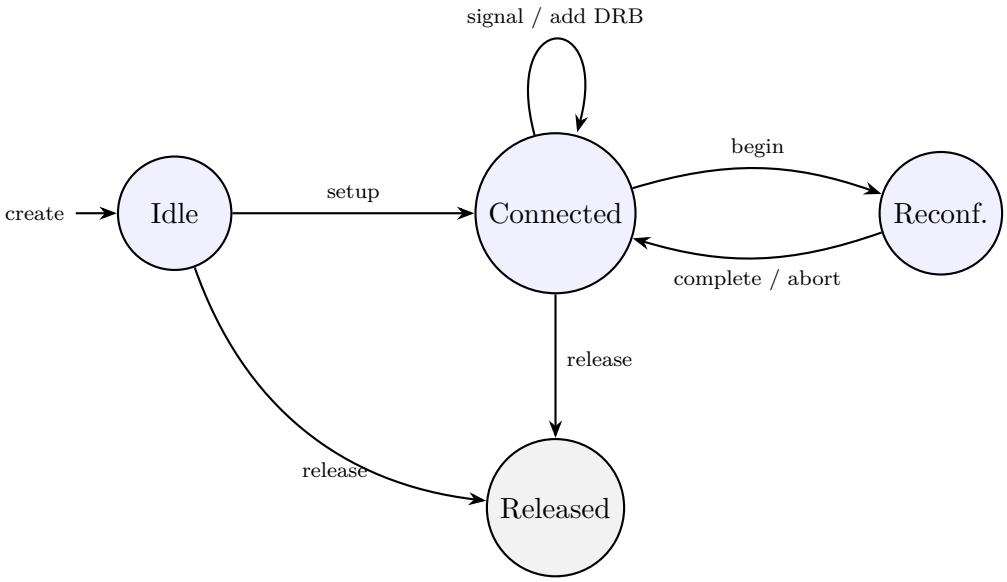


Figure 3.1: UE-context lifecycle state machine. Operations invalid for the current state return an error instead of changing state.

UECM is core RAN layer-3 work. It has the traits that make live control-plane code hard. Each UE has state that changes. The operation lifecycle is strict. Many UEs run at once. Delay targets are the ones the 5G system sets for control-plane procedures [5, 27]. Findings about how a language handles those traits are most relevant to other stateful, multi-thread control-plane services of this kind. The part is also small enough to specify in full and build twice. That is what makes a parity-tested comparison possible.

The interface is small and built around the lifecycle. It can create and release a context. It can set up a connection. It can handle an in-session signalling message. It can add a data radio bearer. It can begin, complete, or abort a reconfiguration. It can look up a context's state under load. Each operation is defined for each state of Figure 3.1. An illegal operation returns a typed error and leaves the context unchanged. That closed set lets the same tests and the same workload drive both versions along the same paths.

RQ2 needs a catalogue, not a loose list of hazards. One is built from this operation set: 14 classes in two groups of seven. Seven come from inside the region safe Rust claims to govern. Seven were chosen because safe Rust is expected not to prevent them. Each has its expectation recorded before the programs were run. Section 5.3.4 reports what each one did.

The tail is the metric because of two traits of this part. One user-visible action, a call setup or a handover, becomes many dependent control-plane operations. The slowest member gates completion. A busy site drives thousands of such procedures per second. So a far-tail point is reached often enough to matter. The failure shape is the second reason. A service of this design tracks its offered load with flat delay until a worker saturates. Then delay jumps sharply in one increment of load. That is why this study sweeps past saturation.

3.2 Research approach

The study runs experiments. It also builds the thing it measures: a matched pair of versions, plus the harness that drives them. Both are built from one frozen spec. Evidence comes from three separate methods. A load sweep answers RQ1 and RQ3. Controlled fault injection answers RQ2. An FFI migration prototype answers RQ4. A weakness in one of them, such as how much the host can move the tail, does not carry into the rest of the claims.

Each question asks how one outcome depends on one variable, the language. The language has to be the only planned gap between the two objects under test. That rules out the most common way to compare languages. In that form two programs written on their own are measured, and the gap is taken to be the language. Such a gap also carries the design, the libraries, and the skill of their authors. Building both sides from one spec removes that confound. Where a question can be answered with numbers, it uses numbers. Where it cannot, the answer is still by a fixed plan, in words. That is how this study treats the kind of guarantee in RQ2. RQ4 asks about the design cost of the application programming interface (API).

The closest prior work is the five vendor-hosted theses of Section 2.3. They appear in no citation database. They had to be found by searching each university's own archive by hand. The same searches found no Finnish thesis on Rust in a radio-access setting. They found no indexed paper mentioning Rust from the vendor hosting this work. The search returned no study that measures the tail of a Rust and a C++ version of the same stateful, multi-thread control-plane part under too much load.

Table 3.1 maps each research question to the method, artefact, data and metric that answer it. RQ1 and RQ3 share H1, worded to cover a speed tie in both the delay and the resource sense. That wording turns out to be the hypothesis's main weakness. RQ2 does not depend on the host. RQ4 depends on it along one axis. Build time is a timing. So it is read as a ratio between two toolchains measured in turn on one machine.

Table 3.1: How each research question is answered. RQ1 and RQ3 carry two methods rather than one: the sweep measures the round-trip a deployment would experience, and the split of the core from the handoff measures the part of it the language is responsible for.

RQ	Method	Artefact / data	Primary metric
RQ1	Load sweep to overload, interleaved repeats	Benchmark harness, latency histograms	median, p99/p99.9/p99.99, and maximum vs. offered load
RQ3	Same sweep, resource capture	`getrusage` samples per run	peak resident set size (RSS), CPU, throughput
RQ1, RQ3	Core called directly, then through the service	Matched microbenchmark pair	operation vs. cross-thread handoff
RQ2	Controlled fault injection	Defect catalogue, three C++ sanitisers, `rustc`, Loom/Shuttle, Miri	compile-time / runtime / undetected
RQ4	FFI migration prototype	C++ host + Rust C-ABI seam	seam size, `unsafe` scope, per-call cost

3.3 Methods per research question, in outline

RQ1 and RQ3 are answered by an open-loop load sweep from light load to well past saturation. Open-loop means the generator keeps to its send schedule instead of waiting for replies [38]. The four build variants are interleaved inside each repeat. A second timing splits the core operation from the cross-thread handoff around it. The end-to-end round-trip bundles two costs of two kinds. Only one of them can differ between the languages. A

third timing takes the contended queue and the reply handle on their own. So the overload cause can be tested and not merely argued.

RQ2 is answered by controlled fault injection [39]. Each defect class is a small program of its own in both languages. The C++ side is built and run under AddressSanitizer. It is also run under ThreadSanitizer and UndefinedBehaviorSanitizer. The safe-Rust side is compiled and, where it compiles, run under a bounded timeout. No static analyser is run, on purpose. The C++ columns then report the tooling this study ran. They do not report a ceiling on what C++ tooling could find. RQ4 is answered by a working prototype in the direction a real migration takes. A C++ host calls an unchanged Rust core across a C ABI. The per-call cost is measured on a real operation and not inferred from an empty crossing. Memory bugs that cross a language sit at that seam [40].

3.4 Fairness as a controlled variable

A language comparison is only as good as the variables it holds fixed. The spec is frozen before either version exists. The thread model, the allocator and its arena count, the opt level, the hash function, the histogram method and the load schedule are matched across the two sides. They are never tuned per side. Where the two languages could not be written alike, the gap is logged instead of resolved. The two sides are shown to match by diffing match logs operation by operation. One row of that table has no control against it. One person wrote both versions and is not equally expert in both. Section 6.5 treats that as the study's main internal threat.

3.5 Statistical treatment

The statistical design is fixed in advance. The family of tested comparisons is listed ahead of the campaign. So a load picked after seeing a sweep cannot enter it. Each comparison is an exact paired permutation test over 20 interleaved repeats, with a bootstrap confidence interval [41], Holm-corrected across the family [42]. The floors the test can reach (0.0000019 uncorrected, 0.000027 corrected) are quoted so that a null can be told apart from a math limit. Two quantities are kept out of the family. One is jitter from one call to the next, which no research question asks for. The other is the pooled p99.99 far tail. Its unit is the operation, not the run. It then has no across-repeat interval to test.

3.6 Reproducibility and research artefacts

A third party can rebuild the results without access to Nokia's private code. The versions are driven entirely by the public frozen spec. The benchmarking harness, the fault catalogue and its runner, and the FFI prototype are published with the thesis. Each plot of measured data is rebuilt by one script from the raw logs. So is each quoted number in the results. The script is `bench/refresh_thesis.sh`. That claim does not cover the hand-drawn schematics, which explain a concept rather than report a result. And running the same scripts over another host's campaign rebuilds the artefacts by the same procedure, not with the same values.

The logs that script reads are part of the package. Each campaign has its own script and its own macro prefix, so a figure from one cannot reach a sentence about another. Each run's host setup is recorded and stored beside the campaign data. Nothing in the results chapters is typed from those files by hand.

4 Design and Implementation

Nothing is built until the spec is frozen. It fixes the public API, the UE state machine, the data layout and the steps on it, the locks and shard size, and the thread model. Both versions follow it. The test then times the language, not a gap of design or of skill. The spec is itself an output of the study. It is the main control against writer bias.

One thread model is used on both sides: shared-nothing and thread-per-shard. That layout is known to change tail delay by itself [43]. The id space is split into shards. Each shard is owned by one worker thread. That thread alone reads or changes it. So no lock guards context state. Callers send a request to the owning shard's bounded queue. They wait on a one-shot reply handle. The queue blocks when it is full. That is on purpose. A drop rule would hide overload. This study needs overload to show up as delay. The only shared mutable state either version allows is one atomic counter. No control flow depends on its value. Figure 4.1 shows the data flow. There is one multi-producer, single-consumer (MPSC) queue per shard. Its worker drains it first in, first out (FIFO).

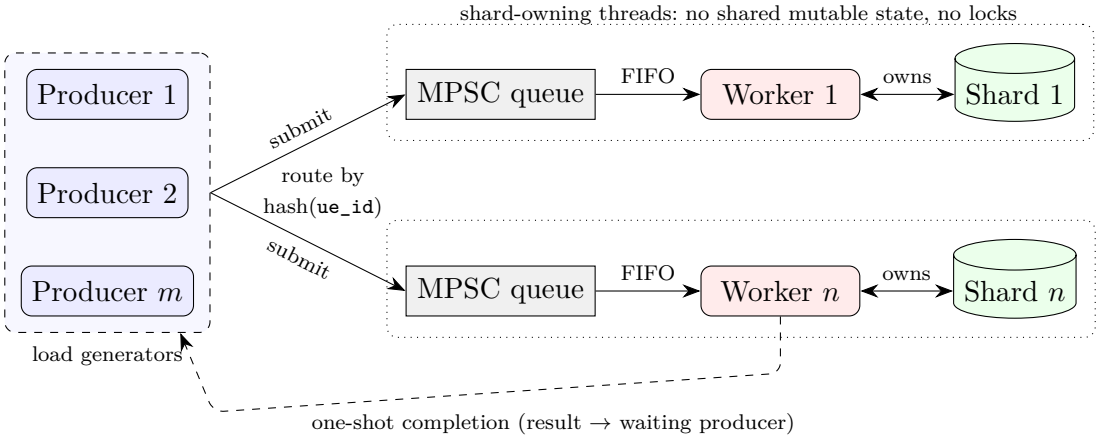


Figure 4.1: Shared-nothing, thread-per-shard architecture. Producers submit to a bounded MPSC queue per shard, and that shard's worker drains it.

Fairness is kept below the design as well as in it. Both builds link the same `jemalloc`. Both keep a system-allocator build as a control. Both compile at the same opt level. Both record delay through one high-dynamic-range (HDR) histogram set up the same way [44]. Hashing is pinned on both sides. SplitMix64 assigns the shard. FNV-1a, the Fowler–Noll–Vo hash, is used inside the map. C++'s default hash of an integer is often just the number itself. Rust's is strong on purpose. Left as they came, the two standard libraries would have been the measurement.

Where the languages cannot meet a need the same way, the gap is written into a forced-idiom log and then measured. Two of those gaps carry the overload result of Section 5.3.2. One is the bounded queue. Rust takes it from `sync_channel`. C++23 still has no such primitive, so C++ builds one from a mutex and two condition variables. The other is the reply handle, `std::promise` against a fresh `mpsc` channel per request. A third carries the core-operation result of Section 5.3.1: the per-shard container is each language's own standard hash map. A fourth is the request encoding. C++ has no payload-carrying enumeration in this idiom, so a request becomes a tagged struct holding each field any operation might need, 160 bytes against Rust's 96. The net result is a Rust core 18% longer in code lines. Its cyclomatic complexity is 15% lower [45]. The Rust library contains no `unsafe` at all.

Parity is tested and not assumed. Language-neutral vectors run in both versions. Their logs are diffed operation by operation. A cross-diff cannot tell "both correct" from "both wrong in the same way". A third version in Python acts as an independent oracle. It has to reproduce all 223 hand-derived expectations before it may generate any of its own. A campaign of 793 000 generated operations found no mismatch. The C++ side runs under AddressSanitizer, ThreadSanitizer and UndefinedBehaviorSanitizer. The Rust side runs under Miri. A clean sanitiser result is bounded by the code that actually ran.

5 Experimental Setup and Evaluation

This chapter starts with the test hosts and the work they run. Results then follow by load, then by the rest of the questions.

5.1 Testbed setup

Some numbers in this chapter come from the tools and the code. Other numbers depend on the host they ran on. Each speed number here comes from one bare-metal host, a single-socket AMD EPYC 7443. Each core runs one thread. Extra threads on a core are off. The host runs Ubuntu 24.04.4 LTS on kernel 6.17.0-23-generic. Linux can raise and lower each core's clock as load and heat change. The clock is held fixed. The governor is performance, and boost is off. The test may use only cores 2–17. There is no hypervisor. The program runs on the host itself, not inside a virtual machine. On those cores the kernel turns off its timer tick (`nohz_full`). If left on, those ticks would show up in the delay numbers. The study does not use `isolcpus`. That flag also stops the kernel sharing work across the allowed cores. Each test thread then lands on one core.

The build tools are not the ones on the host. Both sides are built and run inside an image that carries Rocky Linux 8.10 (Green Obsidian) with `g++ 15.2.1` and `rustc 1.97.1`. The image pins the `gcc-toolset` major and not the point release. The earlier sweep on a virtual machine then built with `g++ 15.1.1`, one GNU Compiler Collection (GCC) 15 release earlier. An image still uses the kernel of the host it sits on. The two sweeps then differ in the hardware, in the kernel (6.17.0-23-generic against 7.0.2-1.el8.elrepo.x86_64), and in that compiler release.

Until late in the work the sweep ran on a virtual machine instead: Kernel-based Virtual Machine (KVM) on an Intel(R) Xeon(R) CPU E5-2680 v4, with none of the core controls above. On that host the 95% band on one arm's p99 spans 1480 μs. The four arms' p99 values differ by 293 μs. The noise on one arm was larger than the gap between all four. That sweep is kept. It uses the same design as this one. The two can then be set next to each other to ask which findings belong to the code. A third sweep on a second bare-metal AMD EPYC 7443P uses a new shard count, load grid, run count and tools. It tests

whether the findings hold under those changes as a set, not any one of them on its own.

5.2 Workload and measurements

The work is made for the test. The steps it takes come from the 3GPP rules [1, 4]. A driver that always sends the same steps walks each UE through the connection lifecycle those rules define. The *signal* work first connects a set of UEs, then sends the same signal call again and again. That picks out the common path. The *mixed* work walks each UE through context creation, setup, signal, bearer setup, reconfiguration or handover, and release. A fixed random order makes both sides see the same counts of each call kind for a given seed. That is also a check that the two sides did the same work. Figure 5.1 shows the two loads and the steps the mixed one walks.

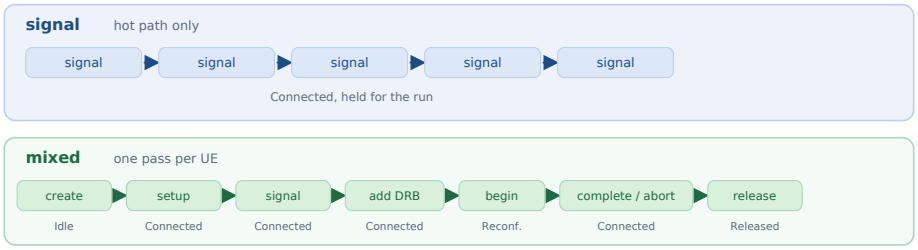


Figure 5.1: The two workloads, and the lifecycle the mixed one walks. Signal repeats the common path, and mixed walks each UE through the procedure sequence drawn below.

The steps match the 3GPP rules. How often each kind shows up does not. How often a real cell hands over rests on the site and the time of day, and no public figure fixes it. A mix that does not match a real cell moves both sides by the same amount. It limits how far the numbers travel. It does not make the test unfair. The sender keeps to its plan even if the service is late (open-loop). The histogram is corrected so a stall does not hide missing replies (coordinated-omission bias) [38]. Each load is run 20 times. The four language and heap arms are interleaved inside each run. Each point is a median with a bootstrap 95% band [41, 46].

The load used for the resource plots is 56 kilo-operations per second (kops/s). It is a chosen value, not one made from the sweep. The first rule this study used was the peak load whose p99 stayed under 5 ms. On the virtual machine that rule landed on the noise of the host. On a quiet host it climbs until the service can no longer keep up. It is kept only as a check. The lowest rate plateau among the four arms is 392 kops/s, reached when the sent load is 392 kops/s.

5.3 Results

The safe code results and most of the migration results do not depend on which host was used. The speed and RSS results do. Each figure below was timed on the host of Section 5.1 unless the caption says so.

5.3.1 Below saturation (RQ1, RQ3)

When the service still keeps up with the sent load, the four arms' medians lie within 1.6 μs of each other, 2.2%. A paired test still finds a gap of 1.3 μs in Rust's favour under both heaps. Figure 5.2 shows the sweep behind that. Higher tails can now be read as well. The widest band on one arm's own p99 spans 0.8 μs. The four arms' p99 values differ by 1.6 μs. So the noise on one is smaller than the gap between them. On the virtual machine the same two sizes were 1480 μs against 293 μs, the other way around. Both p99 rows and both p99.9 rows clear the Holm correction, 4 rows in all, in the same way as the median. Table 5.1 reports p99.9 and the maximum at the chosen load. The maximum is kept out of the tested family, as the pooled p99.99 is.

Table 5.1: Per-operation latency, in microseconds, at 56 kops/s. The median over 20 repeats sits above its bootstrap 95% interval. †The p99.99 column is pooled across repeats, so it has no interval. The maximum is kept out of the tested family for the same reason.

			(µs)		
Variant	p50	p99	p99.9	p99.99†	max
C++ (system alloc)	73.7	76.4	81.2	142.0	243.1
	[73.5, 73.7]	[76.2, 77.0]	[80.8, 81.6]		[239.2, 275.8]
C++ (jemalloc)	73.5	76.6	80.2	176.4	246.3
	[73.3, 73.6]	[76.4, 76.9]	[79.5, 82.0]		[228.8, 259.5]
Rust (system alloc)	72.4	75.0	77.3	92.1	145.2
	[72.3, 72.5]	[74.7, 75.2]	[77.0, 78.7]		[115.9, 178.2]
Rust (jemalloc)	72.1	75.6	77.8	89.5	121.4
	[72.1, 72.3]	[75.6, 75.7]	[77.7, 77.9]		[101.8, 179.6]

The far tail can be read here too. The runs' histograms are pooled, which gives 3 360 051 to 3 360 084 calls per arm. The two Rust arms sit below both C++ arms. The widest pair differs by 87 μs. The slowest Rust arm still sits 50 μs below the fastest C++ one. A second bare-metal host at another shard count agrees, running 147–228 μs for the C++ arms against 88–115 μs for the Rust ones, and the ranges do not meet. A second limit does not come from the host. No host would remove it. An end-to-end round-trip spends

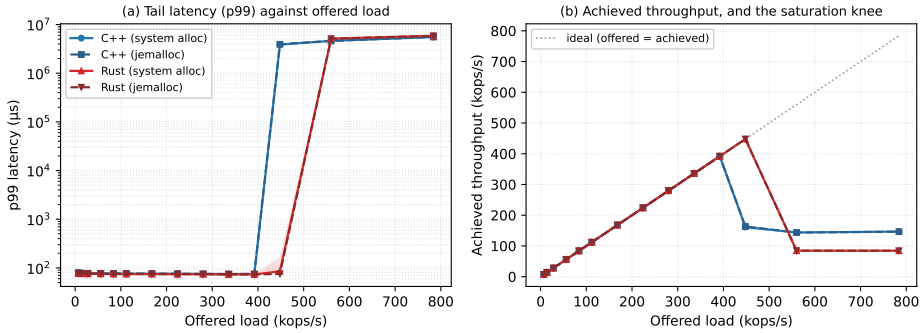


Figure 5.2: The load sweep, both panels against offered load. Lines are medians over 20 repeats, and the bands are bootstrap 95% confidence intervals.

only 0.250% of its time on the work the language does. The rest is passing the request to another thread, and that time is charged to the operating system. When the work inside the language is this small, time spent crossing from thread to thread dominates [47]. This sweep cannot bound a language effect either way, which Figure 5.3 sets beside what the sweep can still show. Timed on its own, the core call is 1.6× faster in Rust. The likely cause is how the stock hash map is laid out, not the safe code model.

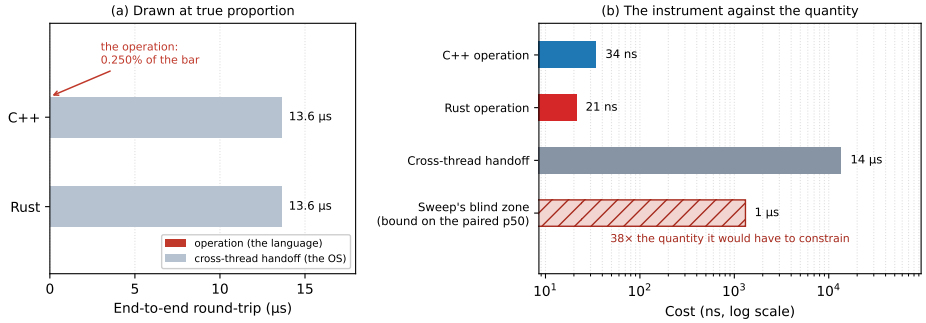


Figure 5.3: Where an end-to-end round-trip's cost goes. **(a)** The round-trip at true proportion. **(b)** Both operation measurements sit inside the sweep's blind zone by a factor of 38.

5.3.2 Past saturation (RQ1)

Once the sent rate is higher than the service can handle, the four arms stop keeping up at loads that do not match. The Rust versions hold to 448 kops/s. The C++ versions hold to 392. So at 448 kops/s Rust still serves 448 kops/s where C++ has fallen to 162. That load is shown and not tested, because it was found after the data was in hand.

Deeper, at 784 kops/s sent, the order flips. C++ still finishes 147 kops/s where Rust finishes 84, a factor of 1.7, in 20/20 runs under both heaps. The Holm-corrected p is as small as this test family can go. The slower arm is not just doing less work. It uses 1.8

times the CPU for each call it finishes, which is the mark of effort spent without finishing anything. When the bounded queue and the done handle are timed by themselves, that order is not significant, so the cause sits in the wider path that passes work between threads. This is the largest language effect in the tested family. A test that stopped at a load the service still meets would have pointed the other way. Figure 5.4 sets what each arm still finishes beside what it spends finishing it.

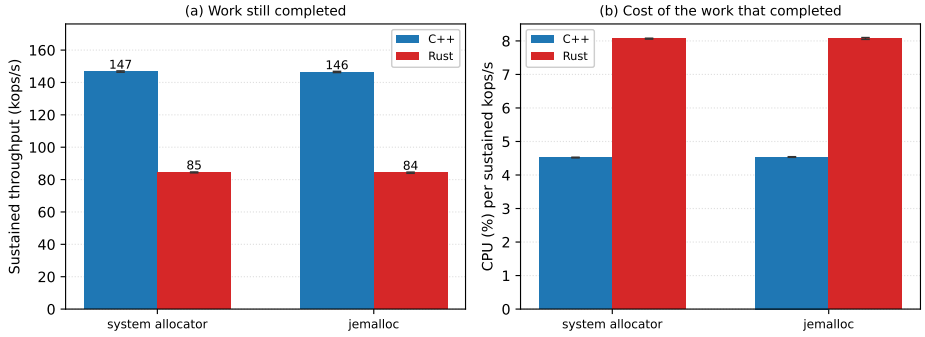


Figure 5.4: Behaviour at 784 kops/s offered. Bars are medians over 20 repeats with bootstrap 95% intervals, grouped so the two bars in a group differ only in language. CPU in (b) is summed across threads.

5.3.3 Resource use (RQ3)

RAM and CPU split the arms where delay could not. Two effects that match in size sit in peak RSS. The Rust build uses 1.4–2.5 MiB more peak RAM than the C++ build under either heap. Linking `jemalloc` at all costs 1.5–2.4 MiB in either language. Both are real, and Figure 5.5 puts the pair beside the peak RSS they add up to. The language effect keeps its sign and size on all three hosts timed. The `jemalloc` extra was 13 times the language effect on the virtual machine this study used first (32.5–34.3 MiB), so their ratio hangs on the host. The language gap is a fixed cost, not a cost per UE. The quiet host is where that can be seen. `jemalloc`'s own counts there put +2.07 MiB more in arena bookkeeping and −0.16 MiB *less* in live bytes. Its resident bytes are −1.92 MiB lower. The process peak is +1.74 MiB higher. The extra RAM then sits past the heap, in the runtime a Rust process carries before it gets RAM. CPU changes sign with the load. At the chosen load of 56 kops/s Rust uses at most 5.1 percentage points less. Once the service no longer keeps up, CPU per finished call is much higher.

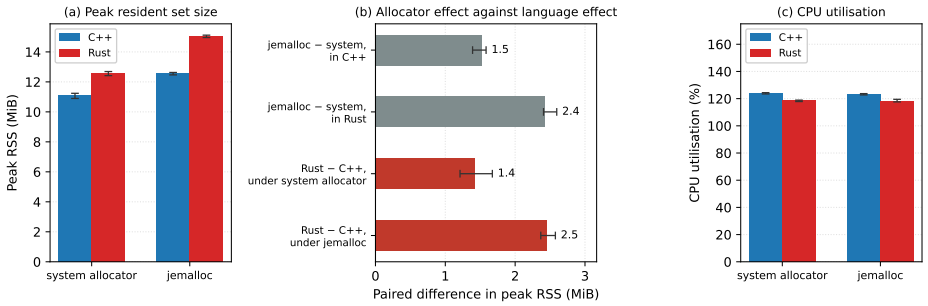


Figure 5.5: Resource use at 56 kops/s, grouped by allocator. Bars are medians over 20 repeats with bootstrap 95% intervals, and CPU in (c) is summed across threads.

5.3.4 Safety via fault injection (RQ2)

Safe code is checked by putting 14 defects in. Seven come from inside the zone Rust claims to govern. Seven were chosen because safe Rust is meant *not* to stop them. Each one of the first seven is a compile error in Rust. In C++ each is valid code. The defect shows up only if a test drives it, and no single sanitiser hits them all. The use-after-move is the case where no sanitiser flags anything. A moved-from object is still a valid object. Its value is unset. So there is nothing wrong to detect, and the name is just lost. All seven boundary cases compile in safe Rust and fail. On two of them the C++ tools are the ones that flag. LeakSanitizer flags the ref cycle that Rust leaks in silence, because leaks are safe in Rust by design [48, 49]. UndefinedBehaviorSanitizer flags the overflow that Rust defines to wrap. ThreadSanitizer catches the lock-order inversion in 5 of 12 runs, a rate that itself jumps. It logs lock orders only from locks that work. If the first inversion waits and never returns, the cycle is never closed, so the tool never flags it.

The seam double-free is the in-guarantee double-free recast through `unsafe`, not a new class. Its C++ half is the same, so only the Rust column moves. Two Rust model checkers do flag both original boundary defects. Loom explores thread orders in a sound way. Shuttle picks the ones Loom cannot reach [50]. Each had to be written using the checker's own parts first, and shrunk to model size. Miri flags the seam double-free, but it reads Rust and cannot run the C++ host the defect sits behind. LeakSanitizer and UndefinedBehaviorSanitizer flag without a rewrite. The two model checkers flag the concurrency boundary cases only after the defect is recast into their parts. Figure 5.6 draws the whole list as an outcome matrix. Resource Acquisition Is Initialization (RAII) removes the plain missed-unlock. It does not remove the class: both forgotten-guard programs hang, and no side's tools said so.

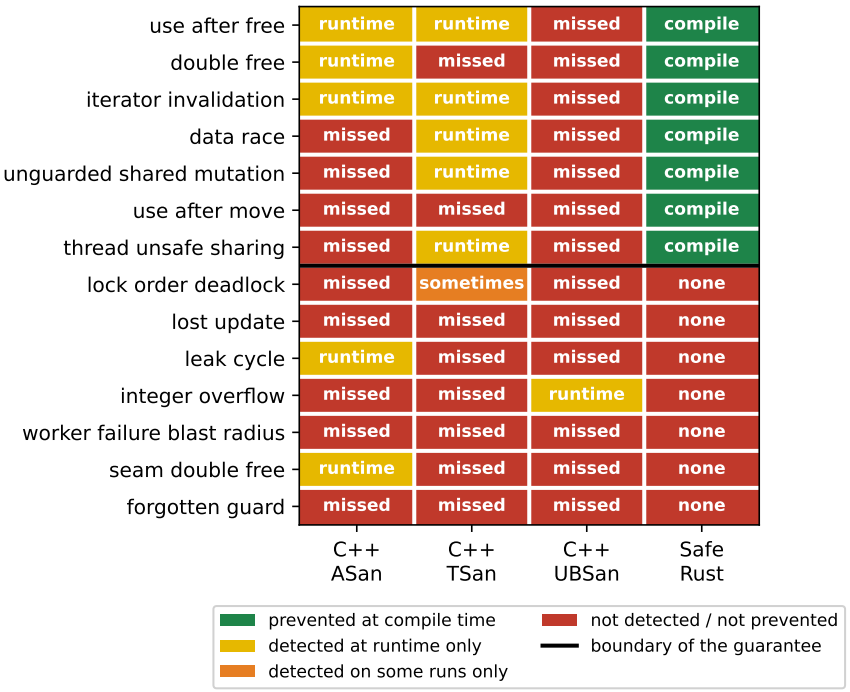


Figure 5.6: The fault-injection outcomes as a matrix, generated from the taxonomy log.

5.3.5 Migration (RQ4)

The move seam is small on each axis timed: 11 C entry points in 231 lines of Rust. All raw-pointer work is kept in 9 functions. The core is left free of `unsafe`. No extra cost is seen against the same signal call made directly. That bounds any per-call tax at 10.91% of that call, and Figure 5.7 shows the structure that bound belongs to. Three adoption costs beyond the seam were measured as well, and two of them go against the usual worry about taking Rust. 54% of the Rust clean build is third-party code. The project's own source builds in 0.8 times the CPU `g++` needs. The 6.8× size gap is a linkage policy that flips when the policy is held fixed. The compared part has no third-party crates and no `unsafe` sites (0 crates, 0 `unsafe` sites).

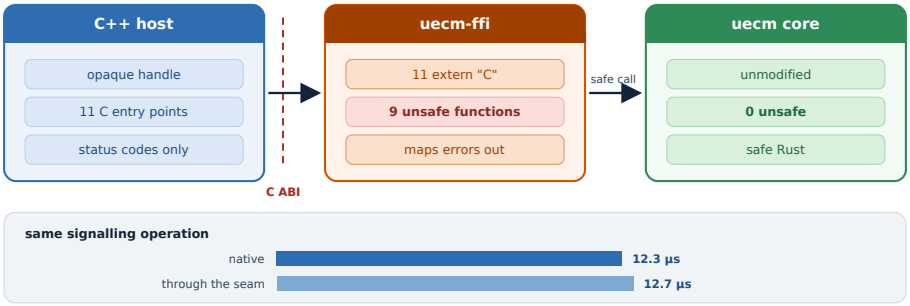


Figure 5.7: The C++-to-Rust FFI migration seam. The same signalling operation costs 12.3 µs directly and 12.7 µs through the seam, a paired difference bounded at 10.91% of that call. A do-nothing crossing costs 2.54 ns and bounds only itself.

6 Discussion

6.1 Summary of findings

The answer to RQ1 splits at saturation. Below it the arms agree to 1.6 μs at the median, and the far tail lands on Rust's side. The end-to-end round-trip is only 0.250% language work, so it cannot bound a language effect. The same core timed apart is 1.6× faster in Rust. Past saturation they split both ways. The Rust arms hold their sent rate to 448 kops/s against 392. At 784 kops/s C++ holds 1.7× the rate of Rust. RQ2 is final and does not depend on the host. All seven in-guarantee classes are compile errors in safe Rust. All seven boundary classes compile and fail in Rust and in C++. RQ3 is a RAM cost of 1.4–2.5 MiB that holds under both heaps and on all three hosts, with CPU that flips across the two load bands. RQ4 is a small seam whose per-call tax is bounded at 10.91% of that call. The real cost is API design and the learning curve.

6.2 Discussion on the research questions

What a live system should do with RQ1 rests on where it expects to sit, not on a single rank. A service sized to run below each arm's knee should pick the arm that holds its sent rate further. One that a signal storm will drive past each knee should pick the arm that still does more work then. Those two picks point at opposite sides. Only the second is tested. The first is shown, not tested, for the reason Section 5.3.2 gives. A test that stopped at a load the service still meets would have read the reverse with the same setup.

The closest prior work times this split for 5G control-plane steps. It finds the same shape and traces it to stackless `async/await` [32]. That claim is accepted here. The shape showed up with no async runtime on either side and the thread model held fixed. A runtime cannot be the only place such an effect starts. The prediction logged in advance also named where to look first, bounds and overflow checking [51, 52]. That part of it was wrong. The median gap fits data that Rust's safe checks are mostly absorbed by modern out-of-order CPUs [12, 35]. It is also weaker than it looks. The core call is 0.250% of what the sweep times.

On safe code some defects are compile errors, so they never become a program that can run. The seven classes Rust does not stop are part of the answer, not a warning

added then. A compiler that will not build a program rules the defect out of each run. A sanitiser rules it out only of the runs a sanitiser test happens to take. What is fixed there is the run coverage, not the breadth of the list. Figure 6.1 names the way on each side of that line. Rust reasons about aliasing and not about lock order. A `Mutex` guard whose life the borrow checker tracks well says nothing about whether two threads take their locks in a matching order. Static deadlock checks for Rust are an open research problem [53]. The lost update misses ThreadSanitizer for the same reason it misses the borrow checker. Where each access is synced right, there is no unsynced access to find. In a field study of 105 real concurrency bugs, about a third of the cases that were not deadlocks broke the order the writers meant, not their locking [54]. Taking Rust drops one class of concurrency defect and leaves in place the class that fails in silence. A plan to move that reads “memory-safe” as “concurrency-safe” would mis-budget its review effort. Memory safety bugs in Rust involve `unsafe`. That holds for each published Common Vulnerabilities and Exposures (CVE) entry but one compiler bug [55]. That is why keeping the core free of `unsafe` matters. It is also why unsafe spreading through crates is a review concern [56].

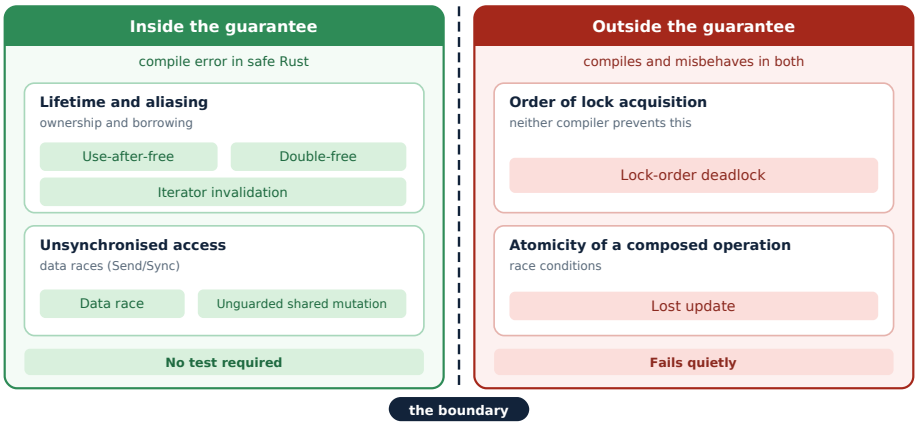


Figure 6.1: Where Rust’s compile-time guarantee stops, with classes grouped by the mechanism that rejects them.

One case reaches past the fault programs. A spec gap let a call be sent outside the service running window. It passed each test, each parity vector and all three sanitisers because nothing ever called that path. What turned undefined behaviour into a catchable failure on the Rust side was a plain bounds check, not a proof. The thing worth planning for is “checked by default”, not “ruled out at compile time”. The root cause belongs to the method. A ban stated without a way behind it leaves each version to fail in its own idiom.

RQ3 splits by resource and the splits do not point the same way. The RAM cost holds under both heaps and on a second host with a new compiler. Peak RSS is a max for the whole run, so the jitter that destroys a p99 does not touch it. Sized against the heap’s own

effect, the two are of the same order on both bare-metal hosts. On the guest the heap's was an order of magnitude larger. The language effect travels. The ratio does not. The quiet host also puts the cost past the heap, by the heap's own counts. One close study reports the other sign, with Rust lowest and the gap growing as concurrency rises [29]. Those two findings agree once the arms are read as two concurrency runtimes, not two sides. Their C++ runs on Boost.Fibers, which gets a stack per fiber. Their Rust runs on Tokio, where a task is a small future on the heap. What they timed is the per-task cost of two concurrency runtimes. Two further studies agree with the way found here [30, 57]. On CPU the sign hangs on the load. A single number for "the CPU cost of Rust's safe code model" would mix two loads that move in opposite ways.

RQ4 comes out cheap in the machine and costly in the people. A C++ host can call an unchanged Rust core across a thin C-ABI seam whose cost on a real signal call is bounded at 10.91% of that call. The main cost is human [58, 59]. Keeping `unsafe` at the seam bounds what has to be checked, not what can be corrupted. Host and part share one address space [60, 61]. The build and footprint axes point the same way as the seam. A vendor worried about either should audit its crate budget before blaming the compiler. The one axis where C++ leads outright is the edit and compile loop for changes that stay inside one source file. The 6.8× size figure sets two linkage policies, and the order flips when the policy is held fixed.

6.3 Revisiting the hypotheses

Section 1.4 set three hypotheses before any run. Saying how they came out is what sets a test apart from a sketch.

H1 The median clause holds in the practical sense. The paired test resolves 1.3 μs, too small to bound a language effect. The far-tail clause does not, and overload, which the wording never named, is where the languages actually part. The reported campaign now resolves that tail, in Rust's favour, and a second bare-metal host agrees on the direction. H1 was right about where a below-saturation difference would show up and wrong that it was the only place. It was also not well posed. "Performance-equivalent" reads as one quantity and names at least three: the delivered round-trip below saturation, the code the compilers emit, and sustained work under overload. Those three do not move together.

H2 For the seven injected classes of its kind it holds without exception. It fails in the looser reading its wording invites. It is accepted for memory safety and data-race

freedom, rejected for concurrency correctness at large, and it is a claim about *safe* Rust only. The seam case shows the same class returning inside `unsafe`.

H3 Any per-call tax is bounded at 10.91% of that signalling call, which is support from a bound on a cost the measurement could have missed, not from the failure to detect one. The accompanying claim, that the real cost is design and training, is supported structurally and not statistically.

6.4 Engineering guidelines for incremental adoption

The rules are the practical form of the findings, and each carries the limit of the finding behind it. Choose a first part for how thin its API is, not for its size. The rewrite of a rich interface is the schedule risk. The edge itself is not. Keep the C ABI in its own crate and the core free of `unsafe`. That keeps unsafe out of the crate chains it spreads through [56]. Leave the concurrency review budget where it is. Prefer a shared-nothing structure in either language. The strongest concurrency result here belongs to the design and to neither compiler. Plan the learning curve as a training cost and track it. Time the part as well as the round-trip. Sweep past the point where the service stops coping. That is the rule this study would most like to have used from the start.

6.5 Where this study sits, and what it cannot support

The studies closest to this one are the five vendor-hosted theses of Section 2.3. Their verdicts clash with each other in each way. The one with the other sign on peak RAM fits this study once its arms are read as two concurrency runtimes, not two sides. Three threats still matter, and the first is that one author wrote both versions. The same failure mode flipped a printed verdict in this field: comparable after a code review and then Rust worse by 85–91% after more speed work [33]. The parity suite bounds what the code does and not speed. The end-to-end clock times the design faithfully and the language hardly at all. The overload split is seen again here with its way cut down and not closed, and no peer-reviewed study reports the same flip.

6.6 Ethical and professional reflection

The study uses no personal data, no subscriber data, and no operator secrets. All traffic is made for the test. The service is based on live code, not copied from it, so the artefacts can be shared in public. The first duty is not to claim too much. A vendor could read the RQ2 result as licence to describe a moved part as “safe”, and the boundary cases are why that reading would be wrong. A deadlocked control plane is an outage. A lost counter update that stays silent can misinform a mobility decision. Nothing shown here stops either. The second duty is how far the data reaches. Tail delay numbers from any single host are a thinner basis for a production roll-out than they look. That is why the earlier guest sweep is kept, not dropped.

7 Conclusion and Future Work

The thesis compared modern C++ and Rust. The object is a UE Context Management service modelled on a real Nokia gNB-CU-CP component. It looked at four dimensions: tail-latency predictability, compile-time defect prevention, safety-model overhead, and cost of adding Rust a piece at a time. Both versions were built against one frozen specification and driven along the same code paths. That is what lets a language effect be told apart from a difference of design or of skill.

On safety the difference is one of kind and it has a clear boundary. Safe Rust rejects all seven injected memory-safety and data-race classes at compile time. Hazards that C++ can only hope to catch under a sanitiser test become programs that cannot be written. All seven boundary classes compile and misbehave in safe Rust as they do in C++. On two of them the C++ tooling reports what Rust's does not. Stated precisely, Rust eliminates data races and aliasing errors. It does not eliminate race conditions, lock-ordering errors, leaks, wrapping arithmetic or the blast radius of a failed worker, and it eliminates nothing inside an `unsafe` block. Where the boundary defects are caught, they are caught at test time. LeakSanitizer and UndefinedBehaviorSanitizer report without a rewrite. The two model checkers report the concurrency cases only after the defect is rewritten into their primitives.

On migration the cost is dominated by API design and the learning curve, not by any run-time tax. An unchanged Rust core can be driven from C++ across a thin seam. The cost on a real signalling call is bounded at 10.91% of that call. All of the `unsafe` sits in the seam, and the core is verified free of it. Most of the Rust build's extra expense is compiling dependencies, not the compiler's own work. The binary-size gap is a linkage policy. The ordering depends on which matched policy is chosen. Where C++ leads is the edit-compile loop, and it leads only for edits that stay inside a translation unit.

On performance the evidence first divides by what is being measured. An end-to-end round-trip here is 0.250% language work. The rest is an operating-system handoff. That clock therefore bounds nothing about either compiler. The same core timed apart is 1.6× faster in Rust. At the representative load of 56 kops/s the paired test resolves 1.3 μs in Rust's favour, too small for a deployment to care about at the median. At p99.99 the slowest Rust variant sits 50 μs below the fastest C++ one. The Rust arms then hold their offered rate to 448 kops/s against 392. Once all four have collapsed, C++ sustains 1.7× the throughput of Rust, which spends 1.8× the CPU on each operation it completes. On

memory the Rust build carries a few MiB more fixed footprint under both allocators, in the same direction and scale on all three machines measured. The allocator choice costs about as much on both bare-metal hosts, and an order of magnitude more on the guest this study used first.

Taken together these support a narrow claim. It is for stateful, concurrent, partitionable control-plane services of this kind. At the loads where the service still meets its offered rate, the choice is which classes of defect a team wants to make unwritable. The other half is whether it will pay the design and training cost of getting them. Past that point it is a speed decision as well. What the thesis adds is an instrument, not a verdict. That instrument is the frozen specification and the parity-tested pair. It is also a timing framework that reaches past the saturation knee. A split of the round-trip says how much of what it times the language can own. A defect catalogue maps where the guarantee stops.

Future work answers the gaps one at a time. The first job needs no new machine: vary the transparent-hugepage policy on one host with everything else held fixed. All three hosts agree that the Rust build carries the larger footprint. They disagree about how that compares against the cost of the allocator. The guest put it an order of magnitude higher, and neither bare-metal host does. The hugepage policy is the candidate this study can name but not test, because it differed between the hosts and only the system-allocator arm inherits it. Beyond that, replaying anonymised production traces would check the operation mix. The framework can also take further control-plane subsystems or the data-heavy layer-2 and physical layer (PHY) path. A subsystem with truly shared mutable state would exercise the two languages differently than a shared-nothing design does. Testing Rust async runtimes against the thread-per-shard model would separate the runtime from the language in the tail result [32]. Timing what in-process isolation of the two heaps costs would price what protecting an `unsafe`-free core is worth [60, 61]. A longer case study tracking defect counts and review effort while adding Rust a piece at a time is what would settle the human cost this study can only name.

Statement on the Use of AI Tools

Tool. Anthropic's Claude Opus 5 and Cursor Grok 4.6 are the generative AI systems used in preparing this thesis. Both ran through the Cursor development environment, from June 2026 through the writing of this thesis. University of Helsinki policy forbids naming a language model as an author. Neither is cited as a source, which is the author's own choice.

What the assistance covered. The models drafted and revised the running prose, and proposed the order of the arguments. They wrote and refactored both versions of the UE Context Management service. They also wrote the load generator, the fault-injection catalogue, and the FFI prototype. The analysis behind the bootstrap intervals and the paired permutation tests is their work too, as are the figure, build and LaTeX sources. The abstract is this programme's maturity test. University policy never permits a language model in one. The abstract was written by the author unaided.

What the models did not settle. The research problem, the four research questions, the three hypotheses, and the study's main control are the author's. That control is one frozen language-neutral specification and one concurrency model, so language remains the only planned gap between the two versions. The author also chose to carry defect classes from outside Rust's compile-time guarantee, which is what keeps RQ2 falsifiable. Every measurement was run by the author, on the machines each result names. The quantities the thesis quotes reach the page as macros rebuilt from the raw logs. The readings are the author's too.

How the output was checked. All model output, prose and code, was read by the author before it entered the thesis. Every bibliography entry was resolved against Crossref, DataCite, or the issuing body. An audit then graded how far each claim could be traced. The first grade is the page or clause itself. The second is a publisher-deposited record that carries the claim. The third is confirmed metadata behind a paywall. An entry whose supporting passage was never opened is listed, not absorbed. The audit found four passages that overstated or misplaced their source. One was a quotation absent from the work it was attributed to. All four were corrected. Code was checked against a conformance suite whose transcripts are diffed line by line. It was also checked against a fault catalogue that fixes each expected outcome before the defect is injected.

Responsibility. The author has reviewed the whole of this thesis and is responsible for

all of it. Where it is wrong, the error is the author's.

Bibliography

[1] 3GPP. *NG-RAN; Architecture description*. Technical Specification (TS) 38.401. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.401/38401-h50.zip.

[2] 3GPP. *NR; NR and NG-RAN Overall description; Stage 2*. Technical Specification (TS) 38.300. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), June 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.300/38300-h50.zip.

[3] M. Polese, L. Bonati, S. D'Oro, S. Basagni, and T. Melodia. "Understanding O-RAN: Architecture, Interfaces, Algorithms, Security, and Research Challenges". In: *IEEE Communications Surveys & Tutorials* 25.2 (2023), pp. 1376–1411. DOI: 10.1109/COMST.2023.3239220. URL: https://doi.org/10.1109/COMST.2023.3239220.

[4] 3GPP. *NR; Radio Resource Control (RRC); Protocol specification*. Technical Specification (TS) 38.331. V17.5.0, Release 17. 3rd Generation Partnership Project (3GPP), July 2023. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.331/38331-h50.zip.

[5] 3GPP. *Study on scenarios and requirements for next generation access technologies*. Technical Report (TR) 38.913. V18.0.0, Release 18. Clause 7.4 sets the control plane latency target, for the transition from a battery-efficient state to the start of continuous data transfer, at 10 ms. 3rd Generation Partnership Project (3GPP), Apr. 2024. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.913/38913-i00.zip.

[6] J. Dean and L. A. Barroso. "The Tail at Scale". In: *Communications of the ACM* 56.2 (2013), pp. 74–80. DOI: 10.1145/2408776.2408794. URL: https://doi.org/10.1145/2408776.2408794.

[7] L. Barroso, M. Marty, D. Patterson, and P. Ranganathan. "Attack of the Killer Microseconds". In: *Communications of the ACM* 60.4 (2017), pp. 48–54. DOI: 10.1145/3015146. URL: https://doi.org/10.1145/3015146.

[8] M. Miller. *Trends, challenge, and shifts in software vulnerability mitigation*. BlueHat IL, Microsoft Security Response Center (MSRC). Approximately 70% of Microsoft CVEs are memory-safety issues. 2019. URL: https://github.com/microsoft/MSRC-Security-Research/tree/master/presentations/2019_02_BlueHatIL.

[9] J. Vander Stoep. *Memory Safe Languages in Android 13*. Google Security Blog. Dec. 2022. URL: https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. 2nd. San Francisco, CA: No Starch Press, 2023. ISBN: 978-1-7185-0310-6. URL: https://openlibrary.org/isbn/9781718503106.

[11] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. "RustBelt: Securing the Foundations of the Rust Programming Language". In: *Proceedings of the ACM on Programming Languages* 2.POPL (2018), 66:1–66:34. DOI: 10.1145/3158154. URL: https://doi.org/10.1145/3158154.

[12] A. Balasubramanian, M. S. Baranowski, A. Burtsev, A. Panda, Z. Rakamarić, and L. Ryzhyk. "System Programming in Rust: Beyond Safety". In: *Proceedings of the 16th Workshop on Hot Topics in Operating Systems (HotOS)*. 2017, pp. 156–161. DOI: 10.1145/3102980.3103006. URL: https://doi.org/10.1145/3102980.3103006.

[13] M. Polese, N. Mohamadi, S. D'Oro, L. Bonati, and T. Melodia. "Beyond Connectivity: An Open Architecture for AI-RAN Convergence in 6G". In: *IEEE Communications Magazine* 64.7 (2026), pp. 90–95. DOI: 10.1109/MCOM.001.2500438. URL: https://doi.org/10.1109/MCOM.001.2500438.

[14] ITU-R. *Framework and overall objectives of the future development of IMT for 2030 and beyond*. Recommendation ITU-R M.2160-0. Approved 13 November 2023. Defines the IMT-2030 ("6G") framework and its development timeline. International Telecommunication Union, Radiocommunication Sector, Nov. 2023. URL: https://www.itu.int/rec/R-REC-M.2160/en.

[15] AI-RAN Alliance. *AI-RAN Platform and Infrastructure Orchestrator*. AI-RAN Alliance, Working Group 2 (AI-and-RAN). Version 10-rc1, clause 2.2. Defines AI-and-RAN as one common IT infrastructure accommodating both AI and RAN workloads, which the infrastructure is required to run concurrently, and notes that the two kinds of workload may have different service-level agreements. Contributors from Ericsson, Nokia, Nvidia, Red Hat, Samsung, SoftBank, zTouch Networks and Northeastern University. Accessed August 2026. Feb. 2026. URL: https://ai-ran.org/documents/AI-RAN-Platform-Infrastructure-Orchestrator.pdf.

[16] J. Vander Stoep. *Rust in Android: Move Fast and Fix Things*. Google Security Blog. Nov. 2025. URL: https://blog.google/security/rust-in-android-move-fast-fix-things/.

[17] A. Rebert and C. Kern. *Secure by Design: Google's Perspective on Memory Safety*. Google Security Engineering, Technical Report. 2024. URL: https://research.google/pubs/secure-by-design-googles-perspective-on-memory-safety/.

[18] R. N. M. Watson, J. Baldwin, D. Chisnall, T. Chen, J. Clarke, B. Davis, N. W. Filardo, B. Gutstein, G. Jenkinson, B. Laurie, A. Mazzinghi, S. W. Moore, P. G. Neumann, H. Okhravi, A. Richardson, A. Rebert, P. Sewell, L. Tratt, M. Vijayaraghavan, H. Vincent, and K. Witaszczyk. "It Is Time to Standardize Principles and Practices for Software Memory Safety". In: *Communications of the ACM* 68.2 (2025), pp. 40–45. DOI: 10.1145/3708553. URL: https://doi.org/10.1145/3708553.

[19] Cybersecurity and Infrastructure Security Agency. *The Case for Memory Safe Roadmaps: Why Both C-Suite Executives and Technical Experts Need to Take Memory Safe Coding Seriously*. Cybersecurity Information Sheet. CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023. URL: https://www.cisa.gov/resources-tools/resources/case-memory-safe-roadmaps.

[20] Office of the National Cyber Director. *Back to the Building Blocks: A Path Toward Secure and Measurable Software*. Technical Report. Washington, DC: The White House, Office of the National Cyber Director (ONCD), Feb. 2024. URL: https://bidenwhitehouse.archives.gov/wp-content/uploads/2024/02/Final-ONCD-Technical-Report.pdf.

[21] European Parliament and Council of the European Union. *Regulation (EU) 2024/2847 on horizontal cybersecurity requirements for products with digital elements (Cyber Resilience Act)*. Official Journal of the European Union, OJ L, 2024/2847. Nov. 2024. URL: https://eur-lex.europa.eu/eli/reg/2024/2847/oj.

[22] Dell'Oro Group. *RAN Mostly Stable in 3Q 2025*. Dell'Oro Group, RAN Quarterly Report summary. The five largest RAN suppliers by worldwide revenue (Huawei, Ericsson, Nokia, ZTE, Samsung) accounted for 96% of the 1Q25–3Q25 RAN market, up from 95% in 2024. Accessed August 2026. Nov. 2025. URL: https://www.delloro.com/news/ran-mostly-stable-in-3q-2025/.

[23] Dell'Oro Group. *RAN Market Stabilized in 2025*. Dell'Oro Group, RAN Quarterly Report summary. Full-year 2025 preliminary findings: supplier rankings unchanged, and overall market concentration measured by the Herfindahl–Hirschman Index

reached a ten-year high. Accessed August 2026. Feb. 2026. URL: https://www.delloro.com/news/ran-market-stabilized-in-2025/.

[24] J. Almodovar and A. Sultan. *The SA1 road to 6G*. 3GPP Highlights, Issue 09. Written by the chair and the secretary of 3GPP WG SA1. Release 20 carries the 6G studies and Release 21 the normative work. ITU-R sets early 2029 for technology proposals and mid-2030 for the submitted specifications. Accessed August 2026. Dec. 2024. URL: https://www.3gpp.org/news-events/3gpp-news/sa1-6g-road.

[25] B. Qin, Y. Chen, Z. Yu, L. Song, and Y. Zhang. "Understanding Memory and Thread Safety Practices and Issues in Real-World Rust Programs". In: *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 2020, pp. 763–779. DOI: 10.1145/3385412.3386036. URL: https://doi.org/10.1145/3385412.3386036.

[26] A. Meneely, A. Green, T. Jaafari, M. Fluet, and B. Keller. ""Just Use Rust": A Best-Case Historical Study of Open Source Vulnerabilities in C". In: *Proceedings of the 3rd IEEE/ACM International Workshop on Software Vulnerability Management (SVM)*. 2025, pp. 25–32. DOI: 10.1109/SVM66695.2025.00008. URL: https://doi.org/10.1109/SVM66695.2025.00008.

[27] 3GPP. *Service requirements for the 5G system; Stage 1*. Technical Specification (TS) 22.261. V18.6.0, Release 18. 3rd Generation Partnership Project (3GPP), Mar. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/22_series/22.261/22261-i60.zip.

[28] 3GPP. *NG-RAN; E1 Application Protocol (E1AP)*. Technical Specification (TS) 38.463. V17.0.0, Release 17. 3rd Generation Partnership Project (3GPP), Apr. 2022. URL: https://www.3gpp.org/ftp/Specs/archive/38_series/38.463/38463-h00.zip.

[29] R. Karlbäck and A. Orö. "Holistic View on Alternative Programming Languages for Radio Access Network Applications in Cloud and Embedded Deployments". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–2021/023–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2021. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-176307.

[30] V. Olsson. "Rust Programming Language as an Alternative to C for RAN Management Applications". Carried out at Ericsson AB. Master's thesis. KTH Royal Institute of Technology, School of Electrical Engineering and Computer Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:kth:diva-325822.

[31] J. Olsson and I. Stenström. "Rust Async for Thread-per-Core Packet Processing". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–23/052–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-197029.

[32] A. Arvidsson and M. Kaller. "Evaluating Asynchronous Programming Frameworks for 5G Control Plane Procedures". Carried out at Ericsson AB. Report LIU-IDA/LITH-EX-A–25/055–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2025. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-214746.

[33] C. Emberg and A. Wadsten. "Attainable Safety and Long-Term Resource-Efficiency Using Rust: Evaluating the Viability of Rust in Terms of Development Effort and Performance for the Decompilation of ASN.1 Messages". Carried out at Ericsson, Linköping. Report LIU-IDA/LITH-EX-A–23/078–SE. Master's thesis. Linköping University, Department of Computer and Information Science, 2023. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:liu:diva-195552.

[34] R. Pereira, M. Couto, F. Ribeiro, R. Rua, J. Cunha, J. P. Fernandes, and J. Saraiva. "Energy Efficiency Across Programming Languages: How Do Energy, Time, and Memory Relate?" In: *Proceedings of the 10th ACM SIGPLAN International Conference on Software Language Engineering (SLE)*. 2017, pp. 256–267. DOI: 10.1145/3136014.3136031. URL: https://doi.org/10.1145/3136014.3136031.

[35] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. "The Case for Writing Network Drivers in High-Level Programming Languages". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–13. DOI: 10.1109/ANCS.2019.8901892. URL: https://doi.org/10.1109/ANCS.2019.8901892.

[36] F. Suchert and J. Castrillon. "STAMP-Rust: Language and Performance Comparison to C on Transactional Benchmarks". In: *Benchmarking, Measuring, and Optimizing (Bench 2022)*. Vol. 13852. Lecture Notes in Computer Science. Springer, 2023, pp. 160–175. DOI: 10.1007/978-3-031-31180-2_10. URL: https://doi.org/10.1007/978-3-031-31180-2_10.

[37] R. Gjomemo, P. H. Phung, E. Ballou, K. S. Namjoshi, V. N. Venkatakrishnan, and L. Zuck. "Leveraging Static Analysis Tools for Improving Usability of Memory Error Sanitization Compilers". In: *Proceedings of the 2016 IEEE International Conference on Software Quality, Reliability and Security (QRS)*. Two of the six authors are

affiliated with Nokia. 2016, pp. 323–334. DOI: 10.1109/QRS.2016.44. URL: https://doi.org/10.1109/QRS.2016.44.

[38] B. Schroeder, A. Wierman, and M. Harchol-Balter. "Open Versus Closed: A Cautionary Tale". In: *Proceedings of the 3rd USENIX Symposium on Networked Systems Design and Implementation (NSDI '06)*. 2006. URL: https://www.usenix.org/conference/nsdi-06/open-versus-closed-cautionary-tale.

[39] R. Natella, D. Cotroneo, and H. S. Madeira. "Assessing Dependability with Software Fault Injection: A Survey". In: *ACM Computing Surveys* 48.3 (2016), 44:1–44:55. DOI: 10.1145/2841425. URL: https://doi.org/10.1145/2841425.

[40] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. "Detecting Cross-Language Memory Management Issues in Rust". In: *Computer Security – ESORICS 2022*. 2022, pp. 680–700. DOI: 10.1007/978-3-031-17143-7_33. URL: https://doi.org/10.1007/978-3-031-17143-7_33.

[41] B. Efron and R. J. Tibshirani. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. ISBN: 978-0-412-04231-7.

[42] S. Holm. "A Simple Sequentially Rejective Multiple Test Procedure". In: *Scandinavian Journal of Statistics* 6 (1979), pp. 65–70. URL: https://www.jstor.org/stable/4615733.

[43] P. Enberg, A. Rao, and S. Tarkoma. "The Impact of Thread-per-Core Architecture on Application Tail Latency". In: *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*. 2019, pp. 1–8. DOI: 10.1109/ANCS.2019.8901874. URL: https://doi.org/10.1109/ANCS.2019.8901874.

[44] G. Tene. *HdrHistogram: A High Dynamic Range Histogram*. Project documentation and reference implementation. Section "Corrected vs. raw value recording calls" documents the `recordValueWithExpectedInterval` correction applied when a response time exceeds the expected interval between requests. Accessed August 2026. URL: https://github.com/HdrHistogram/HdrHistogram.

[45] T. Yin. *Lizard: An extensible cyclomatic complexity analyzer for many programming languages*. Version 1.23.0. Accessed August 2026. 2026. URL: https://github.com/terryyin/lizard.

[46] A. Georges, D. Buytaert, and L. Eeckhout. "Statistically Rigorous Java Performance Evaluation". In: *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*. 2007, pp. 57–76. DOI: 10.1145/1297027.1297033. URL: https://doi.org/10.1145/1297027.1297033.

[47] X. Zhou, V. Leis, X. Yu, and M. Stonebraker. "OLTP Through the Looking Glass 16 Years Later: Communication Is the New Bottleneck". In: *15th Annual Conference on Innovative Data Systems Research (CIDR 2025)*. 2025. URL: https://www.vldb.org/cidrdb/papers/2025/p17-zhou.pdf.

[48] J. Hughes and L. Tratt. "Garbage Collection for Rust: The Finalizer Frontier". In: *Proceedings of the ACM on Programming Languages* 9.OOPSLA2 (2025), pp. 3588–3614. DOI: 10.1145/3763179. URL: https://doi.org/10.1145/3763179.

[49] M. Cui, H. Xu, H. Tian, and Y. Zhou. "rCanary: Detecting Memory Leaks Across Semi-Automated Memory Management Boundary in Rust". In: *IEEE Transactions on Software Engineering* 50.9 (2024), pp. 2472–2484. DOI: 10.1109/TSE.2024.3443624. URL: https://doi.org/10.1109/TSE.2024.3443624.

[50] J. Bornholt, R. Joshi, V. Astrauskas, B. Cully, B. Kragl, S. Markle, K. Sauri, D. Schleit, G. Slatton, S. Tasiran, J. Van Geffen, and A. Warfield. "Using Lightweight Formal Methods to Validate a Key-Value Storage Node in Amazon S3". In: *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles (SOSP)*. 2021, pp. 836–850. DOI: 10.1145/3477132.3483540. URL: https://doi.org/10.1145/3477132.3483540.

[51] Y. Zhang, Y. Zhang, G. Portokalidis, and J. Xu. "Towards Understanding the Runtime Performance of Rust". In: *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2022, 140:1–140:6. DOI: 10.1145/3551349.3559494. URL: https://doi.org/10.1145/3551349.3559494.

[52] S. Harris, S. Cooksey, M. Vollmer, and M. Batty. "Rust for Morello: Always-On Memory Safety, Even in Unsafe Code (Experience Paper)". In: *Proceedings of the 37th European Conference on Object-Oriented Programming (ECOOP)*. Vol. 263. Leibniz International Proceedings in Informatics (LIPIcs). 2023, 39:1–39:27. DOI: 10.4230/LIPIcs.ECOOP.2023.39. URL: https://doi.org/10.4230/LIPIcs.ECOOP.2023.39.

[53] Y. Zhang, K. Zhang, G. Liu, Y. Cai, and S. Qin. "Two Birds One Stone: Effective Static Detection of Resource and Communication Deadlocks in Rust Programs". In: *Automated Software Engineering* 33.2, 65 (2026). Extends the preprint

arXiv:2401.01114. DOI: 10.1007/s10515-026-00614-z. URL: https://doi.org/10.1007/s10515-026-00614-z.

[54] S. Lu, S. Park, E. Seo, and Y. Zhou. "Learning from Mistakes: A Comprehensive Study on Real World Concurrency Bug Characteristics". In: *Proceedings of the 13th International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. 2008, pp. 329–339. DOI: 10.1145/1346281.1346323. URL: https://doi.org/10.1145/1346281.1346323.

[55] H. Xu, Z. Chen, M. Sun, Y. Zhou, and M. R. Lyu. "Memory-Safety Challenge Considered Solved? An In-Depth Study with All Rust CVEs". In: *ACM Transactions on Software Engineering and Methodology* 31.1 (2022), 3:1–3:25. DOI: 10.1145/3466642. URL: https://doi.org/10.1145/3466642.

[56] A. N. Evans, B. Campbell, and M. L. Soffa. "Is Rust Used Safely by Software Developers?" In: *Proceedings of the 42nd International Conference on Software Engineering (ICSE)*. 2020, pp. 246–257. DOI: 10.1145/3377811.3380413. URL: https://doi.org/10.1145/3377811.3380413.

[57] A. Sundvisson and S. Hofer. "User-Space Device Drivers on Embedded Linux: A Comparative Case Study of C and Rust". Bachelor's thesis. Blekinge Institute of Technology, Department of Software Engineering, 2026. URL: https://urn.kb.se/resolve?urn=urn:nbn:se:bth-29775.

[58] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. "Benefits and Drawbacks of Adopting a Secure Programming Language: Rust as a Case Study". In: *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*. USENIX Association, 2021, pp. 597–616. URL: https://www.usenix.org/conference/soups2021/presentation/fulton.

[59] W. Crichton and S. Krishnamurthi. "Profiling Programming Language Learning". In: *Proceedings of the ACM on Programming Languages* 8.OOPSLA1 (2024), pp. 29–54. DOI: 10.1145/3649812. URL: https://doi.org/10.1145/3649812.

[60] E. Rivera, S. Mergendahl, H. Shrobe, H. Okhravi, and N. Burow. "Keeping Safe Rust Safe with Galeed". In: *Proceedings of the 37th Annual Computer Security Applications Conference (ACSAC)*. 2021, pp. 824–836. DOI: 10.1145/3485832.3485903. URL: https://doi.org/10.1145/3485832.3485903.

[61] P. Kirth, M. Dickerson, S. Crane, P. Larsen, A. Dabrowski, D. Gens, Y. Na, S. Volckaert, and M. Franz. "PKRU-Safe: Automatically Locking Down the Heap Between Safe and Unsafe Languages". In: *Proceedings of the 17th European Conference on*

Computer Systems (EuroSys). 2022, pp. 132–148. DOI: 10.1145/3492321.3519582. URL: https://doi.org/10.1145/3492321.3519582.